Minimum Area Rectangle

AlgoExpert · Very Hard · Arrays

The problem

- Smallest-area axis-aligned rectangle whose 4 corners are in the point set, else 0.
- Treat each pair as a diagonal and check the other two corners.

Intuition

- Any two points that could be opposite corners determine the other two, so check them in a hash set.

Brute force vs optimal

- All 4-point combos → $O(n^4)$
- Pairs as diagonals + hash set → $O(n^2)$
- Or group by columns

Approach

- 1. Put points in a set
- 2. For each pair with different x and y, check the other corners
- 3. Track the smallest area

Worked example

```
[(1, 5), (5, 1), (4, 2), (2, 4), (2, 2), (1, 2), (4, 5), (2, 5), (-1, -2)]
```

Points

Worked example

```
diagonal (1, 5) – (2, 2)

need (1, 2) and (2, 5): both present ✓
```

Treat each pair as a diagonal

Worked example

```
area 1 × 3 = 3
```

Smallest area: 3

Java solution

```java
public class MinimumAreaRectangle {
    public static int minimumAreaRectangle(int[][] points) {
        Set<Long> set = new HashSet<>();
        for (int[] p : points) set.add(key(p[0], p[1]));
        long best = Long.MAX_VALUE;
        for (int i = 0; i < points.length; i++)
            for (int j = i + 1; j < points.length; j++) {
                int[] a = points[i], b = points[j];
                if (a[0] == b[0] || a[1] == b[1]) continue;
                if (set.contains(key(a[0], b[1])) && set.contains(key(b[0], a[1])))
                    best = Math.min(best, (long) Math.abs(a[0] - b[0]) * Math.abs(a[1] - b[1]));
            }
        return best == Long.MAX_VALUE ? 0 : (int) best;
    }

    private static long key(int x, int y) { return ((long) x << 32) ^ (y & 0xffffffffL); }
}
```

Complexity

- Time: O(n^2)
- Space: O(n)

Edge cases & pitfalls

- Skip pairs sharing x or y
- None → 0
- Use long for area

Interview tips & follow-ups

- Mention column-pair alternative

Recap

- Any two points that could be opposite corners determine the other two, so check them in a hash set.
- Complexity: O(n^2)